

TaskOrbit Design Documentation

System Overview

TaskOrbit is a real-time voice AI platform built around a modular orchestration engine. Rather than encoding agent behaviour in a monolithic system prompt, the system uses JSON-based Task Definitions to describe each agent's workflow, required inputs, and available tools. At runtime, a Python orchestration engine drives each conversation step by step, loading only the context relevant to the current moment. This prevents instruction drift as conversations grow longer, and makes agent behaviour auditable, testable, and changeable without modifying application code.

Runtime Architecture

The system runs as two separate backend processes:

1. **FastAPI HTTP API (port 8000)** — handles REST requests: conversation turns, agent configuration management, LiveKit token minting, TTS synthesis, and health checks. Stateless per request.
2. **LiveKit Voice Worker** — a long-running process that joins LiveKit rooms and runs the full voice pipeline: STT via Deepgram, orchestration, LLM inference, TTS via ElevenLabs. Audio flows over WebRTC — the HTTP API never handles audio bytes.

Browser (React + LiveKit JS SDK)

```
|
+-- HTTP/JSON --> FastAPI backend (:8000)
|                               |-- REST: conversations, agent configs, tokens, TTS
|
|-- WebRTC audio --> LiveKit Cloud --> Voice Worker
                                   +-- Deepgram STT
                                   +-- Orchestration Engine
                                   +-- LLM (OpenAI / Gemini)
                                   |-- ElevenLabs TTS
```

The orchestration engine is the only stateful component. It holds session state — current workflow step, collected data, conversation history — and decides what context to pass to the LLM at each turn.

Code Structure

backend/src/taskorbit/	
+-- api/	HTTP routes (conversations, livekit, tts, agent configs)
+-- orchestration/	ConversationOrchestrator - the core engine
+-- agents/	BaseAgent, SalesAgent, TechnicalSupportAgent, AgentRegistry
+-- tools/	data_extraction, end_call, agent_transfer
+-- integrations/llm/	LLMClient protocol, OpenAI and Gemini clients, factory
+-- livekit_agent/	AgentSession, OrchestratorAgent, voice worker entry point
+-- database/	SQLAlchemy models, repositories, Alembic migrations
+-- logging/	structlog configuration
+-- observability/	Prometheus metrics, Grafana dashboards, Loki log aggregation
+-- config.py	Centralised settings via pydantic-settings

`-- types.py` Shared Pydantic domain types

api/ — FastAPI routes. The conversations route persists messages automatically via `ConversationOrchestrator._persist_messages()`. The agent configs route provides full CRUD for saved agent configurations backed by PostgreSQL.

orchestration/ — `ConversationOrchestrator.process_message()` is the main entry point. It detects intent, selects an agent, selects the active tool, builds a minimal system prompt, calls the LLM, and returns a `ConversationResponse`.

integrations/llm/ — Provider-agnostic `LLMClient` Protocol. Concrete clients: `OpenAIClient` (openai SDK) and `GeminiClient` (google-genai SDK). A factory function `get_llm_client()` reads `agent_config.llm.provider` — provider selection is per-task, not global.

livekit_agent/ — `AgentSession` wires Deepgram STT, ElevenLabs TTS, and Silero VAD. `OrchestratorAgent` overrides `llm_node()` to route inference through `ConversationOrchestrator` instead of the LLM plugin directly. Push-to-talk is enforced via a `reply_requested` flag — the session only generates a reply when explicitly triggered.

database/ — SQLAlchemy async ORM. Tables: `conversations`, `conversation_messages`, `tool_executions`, `users`, `agent_configurations`. Alembic manages migrations.

observability/ — Prometheus metrics at `/metrics` (port 8000) and `/metrics` (port 8001 for the worker). Loki aggregates structured logs from both processes. Grafana dashboards are auto-provisioned.

Agent Task Schema

Every agent is described by a single JSON file conforming to `agent-task.schema.json` (JSON Schema draft 2020-12, version 1.0.0). Three top-level fields are `schema_version`, `task`, and `agent`.

The `task` block defines: `name`, `description`, `required_inputs` (fields the agent must collect, each with `name`, `type`, `required`, `description`), and `workflow_steps` (ordered steps with `id`, `action`, and an optional tool reference).

The `agent` block defines: `agent_id`, `metadata` (`display_name`, `persona`), `instruction` (the system prompt), `first_message`, `variables` (session-scoped with defaults), `pipeline` (STT/LLM/TTS provider config), and `tools` (callable actions with typed parameters).

Reserved workflow step actions: `send_first_message` returns `agent.first_message` directly without an LLM call; `end_call` terminates the session. All other actions delegate to the LLM with the current conversation history.

Technology Stack

Area	Technology	Version	Role
Frontend	React + Vite	19 / 5	Web UI and LiveKit client
Backend	Python + FastAPI	3.11 / latest	HTTP API and orchestration

Area	Technology	Version	Role
Voice	LiveKit + LiveKit Agents	latest	WebRTC audio transport and agent session management
STT	Deepgram (nova-3)	latest	Speech-to-text
LLM	OpenAI or Google Gemini	gpt-4o-mini / gemini-2.5-flash	Reasoning and response generation
TTS	ElevenLabs (eleven_multilingual_v2)	latest	Text-to-speech, 29 languages
Database	PostgreSQL 16	16	Persistent storage
ORM	SQLAlchemy (async) + Alembic	2.x	ORM and migrations
Deployment	Google Cloud Platform	—	Cloud Run, Cloud SQL, Artifact Registry
Containerisation	Docker + docker-compose	—	Local and production containers
Logging	structlog + Loki + Grafana	—	Structured logging and observability

Key Architecture Decisions

JSON-based task orchestration over monolithic prompts — Each agent workflow is described by a versioned JSON file rather than embedded in code or a giant prompt. This makes agent behaviour auditable, testable, and changeable without touching application code. The orchestration engine loads only the context for the current workflow step, which prevents instruction drift as conversations progress.

Python as the backend language — LiveKit’s Python Agents SDK is the primary supported path for building voice agent workers; the Node.js SDK is less mature for this use case. Pydantic v2 validates JSON task definitions directly against Python models, eliminating a separate validation library. The LLM and STT/TTS ecosystems are predominantly Python-first.

Per-task LLM provider selection — The LLM provider and model are declared in each agent’s task definition rather than set globally. This allows different agents in the same deployment to use different providers or models. Adding a new provider requires writing one client class and one factory branch — no orchestration changes.

Two-process backend — The HTTP API (FastAPI) and the voice worker (LiveKit Agents) run as separate processes with independent lifecycles. The API is stateless and scales horizontally; the worker holds a persistent LiveKit connection and scales independently. Merging them would couple two incompatible execution models.

Structured logging across both processes — Both the HTTP API and the voice worker emit JSON-structured logs to stdout via structlog. Promtail ships logs to Loki; Grafana queries both processes in a single dashboard. Token usage and pipeline latency are tracked across both the REST path and the voice path using Loki LogQL over the `llm_call_completed` event.